1 Background

Malicious software, otherwise known as malware, are widely considered to be one of the most pervasive dangers faced by users in the current digital threat landscape [6]. In order to mitigate this risk, detection of such programs is of the utmost importance to security researchers, and has been an active area of research within the field [12]. In this section, we explore some of the defining characteristics of malware, how authors of these programs evade existing detection methods, and a number of proposed approaches that leverage machine learning to detect this form of software.

1.1 Malware

Malware are defined as programs which perform unwanted or harmful actions on a host system without the consent or knowledge of its users. The actions of these programs on an infected system vary, but they all have an adverse impact on the confidentiallity, integrity, and availability of a system. These properties comprise the core tennant of information security, known as the CIA triad, which guide policies for ensuring the security of a computer system. Violation of this principle can provide hostile actors access to sensitive information, the ability to modify data, or the disruption of a systems regular operation, all without necessary authorisation.

Security researchers often classify these programs based on their method of propagation and actions on a host upon infection. It is worth noting however, that the categories within this taxonomy are not mutually exclusive, and modern malware often exist in multiple categories at once.

- **Worm** — Malicious software which propagate throughout a network of interconnected systems.
- **Virus** — Programs that modify other programs in order to replicate throughout an infected system.
- **Trojans** — Malware that disguises itself as legitimate software in order to infect its host.
- **Backdoor** — Software that provide covert remote access to infected systems.
- **Ransomware** — Malware that deny access to systems or information until a fee is paid.
- **Information Stealer** — Malware which automatically extract information, such as credentials, and the activities of users.
- **Botnet** — Often part of a larger network of infected systems, controlled by a threat actor to perform large scale attacks against digital infrastructure.
- **Rootkit** — Malware providing privileged access to an infected system.

1.2 Obfuscation

Commercial anti-malware services utilise a set of heuristics of features that are indicative of malware. An example of such a hueristic could be the presence of a static string, or a series of bytes at a given offset within a program. A large collection of these hueristics are maintained by security researchers, which provides an efficient and accurate means to detect malware.

In order to evade detection by these hueristics, malware authors utilise a set of techniques known as obfuscation. These techniques modify the structure of the program, while preserving its intended behavior. This is a common feature of malware, as it effectively tricks these detection heuristics, without affecting its behavior. In the following sections, we explore some of the obfuscation techniques commonly employed by threat actors.

1.2.1 Encrypted Malware

Obfuscation through encryption is an technique which utilises cryptographic algorithms to hide the true functionality of a program. Malware that employ this form of obfuscation are typically comprised of two parts, the encrypted malicious section and its' decryptor [23]. Upon running the program, the decryptor will recover and then execute the malicious sections. The key used for decryption is changed across instances of the program, which further impact the ability of malware scanners to detect programs employing this form of obfuscation. However, the decryptor portion will remain constant across instances.

Oligomorphic malware extends upon this idea, by modifying the decryptor portion of the program to further confuse detection software. This enables to evade detection far more effectively than encryption alone [24].

1.2.2 Polymorphic Code

Polymorphic code refers to a program which modifies its own instructions, but performs the same operations. Malware utilising this form of obfuscation mutate their structure upon each new infection. The mutations made to the program could include randomly re-assigning registers, inserting dead code and data, among other more advanced techniques. This form of obfuscation is incredibly powerful, as existing detection engines would struggle to find patterns between instances of polymorphic code, making detection extremely difficult.

1.2.3 Dead Code Insertion

One of the less sophisticated forms of obfuscation is to simply insert instructions which have no purpose. This could include no-operations are skipped by the CPU, or performing useless operations, such as modifying unused registers.

1.3 Detection

A number of approaches to malware detection have been proposed in the literature. These largely fall into three categories, based on how features are extracted for later use for classification, these are as follows:

- **Static** — Features extracted from the structure of a program, including metadata, instructions, and data found within the executable.
- **Dynamic** — Features extracted during the exectution of the program, including external function calls, communication, and modifications made to the host.
- **Hybrid** — A combination of both *static* and *dynamic* features

1.3.1 Static

Static detection methods leverage features derrived from the structure of the program rather than its behavior during execution. Features used by this form of detection seen in the literature include sequences of instructions executed by the CPU [2, 16], printable strings [19], external functions and libraries used by the program [17], and the executable header [15]. This form of malware detection is known to be more efficient than dynamic and hybrid methods, as it does not require the use of a Virtual Machine to safely execute a program in order to observe its behavior. However, it can be less robust to obfuscation than other approaches.

In [17], the authors utilise features including static strings, external libraries and functions used within by a program to detect new malicious programs. They train a number of conventional machine learning algorithms, such as Naive Bayes, Multi-Naive Bayes, and RIPPER on these features. The authors were able to achieve 97% classification accuracy with the Multi-Naive Bayes classifier trained on static strings. However, the models trained on libraries and function calls performed significantly worse, indicating these features are unsuitable for reliable detection. Furthermore, the dataset used by the authors of this research was unbalanced, with 3265 malicious, and only 1001 benign samples.

A great deal of previous research into static malware detection utilises a Convolutional Neural Network (CNN), trained on graphical representation of programs [21, 13, 8, 12]. These have shown to be a promising approach to this problem, as CNNs are effective at learning classification problems [7]. However, as these models require an input of fixed size, it is necessary to rescale the images to a fixed resolution before they can be used by the model. Resizing images is a lossy operation, and the size of executables in both categories varies across samples, so important features used by these models may be lost depending on the size of the program, effecting its classification accuracy.

Much of the literature that consider opcode sequences for malware detection focus on its use in traditional machine learning algorithms [2, 16]. These approaches both make use of Hidden Markov Models, and achieve promising results, indicating these features are a viable option for detecting malware. Extending upon this idea, [4] outlines an approach in which the authors propose the use of both stacked bidirectional Long-Short Term Memory (BiLSTM) models and pretrained language models, such as GPT-2. In their investigation, the authors found that their models based on DistilBERT were able to detect malware, with 98.3% accuracy. However, in their investigation, they made use of pre-trained langauge models, which are unlikely to have an understanding of machine intructions.

We have also seen other approaches to malware detection utilising language models such as bert [14], that are fine tuned on the metatdata found within an applications manifest file. This includes information regarding the permissions, activities, and version of the application. Once trained, the authors model was able to detect malware with a high accuracy of 97%. However, their approach is limited to the domain of android malware detection, as other operating systems do not make use of manifest files.

1.3.2 Dynamic

Dynamic malware detection refer to detection systems trained on features extracted from a program during its execution. Common behavioral features seen in the literature include API call sequences [9, 1, 3, 10], changes made to the file system [18, 20], and network communications made by the program [22].

Much of the research in this area makes use of API call sequences during a processes execution. This provides the various approaches that utilise these features with an understanding of changes made to a host system by a program in a concise manner. Detection models that make use of these features have been shown to be an effective means by which to detect malware. For example, in [9], the authors propose a signature based model, based on sequences of API calls that have undergone multi-sequence alignment. Their approach does not make use of any machine learning or neural network based models, yet it is able to reliably detect malware. As malware often perform similar operations, the sequences of API calls will be similar across instances, regardless of any obfuscations made to the program structure.

In their 2022 paper, Li et al. [10], also make use of API call sequence information to detect malware. They extend upon existing research by introducing the arguments given to APIs into the sequence information, in the form of a graph, where each function is represented as a node, and where shared arguments between consequitive API calls are given as edge data. The authors then design and train a graph neural network on this

data, which is able to effectively detect malware, with an accuracy of 98%.

However, some forms of malware obfuscate API calls effectively defeating this approach to malware detection [11]. As such, it is not a reliable feature for malware detection and it is for this reason that other approaches have been explored. In [5], the authors outline an approach that again makes use of graph neural networks trained on behavioral profiles of advanced malware, such as those developed by Advanced Persistent Threats (APT). The behavioral profile consider several aspects relating to system events, including events occuring with regards to processes, the system registry, network connections, and file modifications.

1.3.3 Hybrid

References

[1] Mamoun Alazab. "Profiling and Classifying the Behavior of Malicious Codes". In: *Journal of Systems and Software* 100 (Feb. 1, 2015), pp. 91–102. ISSN: 0164-1212. DOI: 10.1016/j.jss.2014.10.031. URL: https://www.sciencedirect.com/science/article/pii/S0164121214002283 (visited on 02/25/2025).

[2] Srilatha Attaluri, Scott McGhee, and Mark Stamp. "Profile Hidden Markov Models and Metamorphic Virus Detection". In: *Journal in Computer Virology* 5.2 (May 1, 2009), pp. 151–169. ISSN: 1772-9904. DOI: 10.1007/s11416-008-0105-1. URL: https://doi.org/10.1007/s11416-008-0105-1 (visited on 02/25/2025).

[3] Zhi-Guo Chen et al. "Automatic Ransomware Detection and Analysis Based on Dynamic API Calls Flow Graph". In: *Proceedings of the International Conference on Research in Adaptive and Convergent Systems*. RACS '17. New York, NY, USA: Association for Computing Machinery, Sept. 20, 2017, pp. 196–201. ISBN: 978-1-4503-5027-3. DOI: 10.1145/3129676.3129704. URL: https://dl.acm.org/doi/10.1145/3129676.3129704 (visited on 02/25/2025).

[4] Denız Demırcı et al. "Static Malware Detection Using Stacked BiLSTM and GPT-2". In: *IEEE Access* 10 (2022), pp. 58488–58502. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2022.3179384. URL: https://ieeexplore.ieee.org/document/9785789?utm_source=chatgpt.com (visited on 02/25/2025).

[5] Cho Do Xuan and given-i=DT family=Huong given=DT. "A New Approach for APT Malware Detection Based on Deep Graph Network for Endpoint Systems". In: *Applied Intelligence* 52.12 (Sept. 1, 2022), pp. 14005–14024. ISSN: 1573-7497. DOI: 10.1007/s10489-021-03138-z. URL: https://doi.org/10.1007/s10489-021-03138-z (visited on 02/18/2025).

[6] European Union Agency for Cybersecurity. *ENISA Threat Landscape 2024: July 2023 to June 2024*. LU: Publications Office, 2024. URL: https://data.europa.eu/doi/10.2824/0710888 (visited on 02/21/2025).

[7] Omar Habibi, Mohammed Chemmakha, and Mohamed Lazaar. "Performance Evaluation of CNN and Pre-trained Models for Malware Classification". In: *Arabian Journal for Science and Engineering* 48.8 (Aug. 1, 2023), pp. 10355–10369. ISSN: 2191-4281. DOI: 10.1007/s13369-023-07608-z. URL: https://doi.org/10.1007/s13369-023-07608-z (visited on 02/25/2025).

[8] M. Jain, W. Andreopoulos, and M. Stamp. "Convolutional Neural Networks and Extreme Learning Machines for Malware Classification". In: *Journal of Computer Virology and Hacking Techniques* 16.3 (2020), pp. 229–244. DOI: 10.1007/s11416-020-00354-y.

[9] Youngjoon Ki, Eunjin Kim, and Huy Kang Kim. "A Novel Approach to Detect Malware Based on API Call Sequence Analysis". In: *International Journal of Distributed Sensor Networks* 11.6 (June 1, 2015), p. 659101. ISSN: 1550-1329. DOI: 10.1155/2015/659101. URL: https://doi.org/10.1155/2015/659101 (visited on 11/02/2024).

[10] Ce Li et al. "DMalNet: Dynamic Malware Analysis Based on API Feature Engineering and Graph Learning". In: *Computers & Security* 122 (Nov. 1, 2022), p. 102872. ISSN: 0167-4048. DOI: 10.1016/j.cose.2022.102872. URL: https://www.sciencedirect.com/science/article/pii/S0167404822002668 (visited on 11/27/2024).

[11] Yang Li et al. "APIASO: A Novel API Call Obfuscation Technique Based on Address Space Obscurity". In: *Applied Sciences* 13.16 (16 Jan. 2023), p. 9056. ISSN: 2076-3417. DOI: 10.3390/app13169056. URL: https://www.mdpi.com/2076-3417/13/16/9056 (visited on 02/25/2025).

[12] Pascal Maniriho, Abdun Naser Mahmood, and Mohammad Jabed Morshed Chowdhury. "A Systematic Literature Review on Windows Malware Detection: Techniques, Research Issues, and Future Directions". In: *Journal of Systems and Software* 209 (Mar. 1, 2024), p. 111921. ISSN: 0164-1212. DOI: 10.1016/j.jss.2023.111921. URL: https://www.sciencedirect.com/science/article/pii/S0164121223003163 (visited on 02/18/2025).

[13] Sang Ni, Quan Qian, and Rui Zhang. "Malware Identification Using Visualization Images and Deep Learning". In: *Computers & Security* 77 (Aug. 1, 2018), pp. 871–885. ISSN: 0167-4048. DOI: 10.1016/

j.cose.2018.04.005. URL: https://www.sciencedirect.com/science/article/pii/S0167404818303481 (visited on 02/21/2025).

[14] Abir Rahali and Moulay A. Akhloufi. "MalBERT: Malware Detection Using Bidirectional Encoder Representations from Transformers". In: *2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC). Oct. 2021, pp. 3226–3231. DOI: 10.1109/SMC52423.2021.9659287. URL: https://ieeexplore.ieee.org/abstract/document/9659287 (visited on 02/25/2025).

[15] Vinayakumar Ravi et al. "A Multi-View Attention-Based Deep Learning Framework for Malware Detection in Smart Healthcare Systems". In: *Computer Communications* 195 (Nov. 1, 2022), pp. 73–81. ISSN: 0140-3664. DOI: 10.1016/j.comcom.2022.08.015. URL: https://www.sciencedirect.com/science/article/pii/S0140366422003231 (visited on 02/25/2025).

[16] Neha Runwal, Richard M. Low, and Mark Stamp. "Opcode Graph Similarity and Metamorphic Detection". In: *Journal in Computer Virology* 8.1 (May 1, 2012), pp. 37–52. ISSN: 1772-9904. DOI: 10.1007/s11416-012-0160-5. URL: https://doi.org/10.1007/s11416-012-0160-5 (visited on 02/25/2025).

[17] M.G. Schultz et al. "Data Mining Methods for Detection of New Malicious Executables". In: *Proceedings 2001 IEEE Symposium on Security and Privacy. S&P 2001*. Proceedings 2001 IEEE Symposium on Security and Privacy. S&P 2001. May 2001, pp. 38–49. DOI: 10.1109/SECPRI.2001.924286. URL: https://ieeexplore.ieee.org/document/924286 (visited on 02/21/2025).

[18] Daniele Sgandurra et al. *Automated Dynamic Analysis of Ransomware: Benefits, Limitations and Use for Detection*. Sept. 10, 2016. DOI: 10.48550/arXiv.1609.03020. arXiv: 1609.03020 [cs]. URL: http://arxiv.org/abs/1609.03020 (visited on 02/25/2025). Pre-published.

[19] P. V. Shijo and A. Salim. "Integrated Static and Dynamic Analysis for Malware Detection". In: *Procedia Computer Science*. Proceedings of the International Conference on Information and Communication Technologies, ICICT 2014, 3-5 December 2014 at Bolgatty Palace & Island Resort, Kochi, India 46 (Jan. 1, 2015), pp. 804–811. ISSN: 1877-0509. DOI: 10.1016/j.procs.2015.02.149. URL: https://www.sciencedirect.com/science/article/pii/S1877050915002136 (visited on 02/25/2025).

[20] Jan Stiborek, Tomavs Pevny, and Martin Rehak. "Multiple Instance Learning for Malware Classification". In: *Expert Systems with Applications* 93 (Mar. 1, 2018), pp. 346–357. ISSN: 0957-4174. DOI: 10.1016/j.eswa.2017.10.036. URL: https://www.sciencedirect.com/science/article/pii/S0957417417307170 (visited on 02/25/2025).

[21] Jiawei Su et al. "Lightweight Classification of IoT Malware Based on Image Recognition". In: *2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC)*. Vol. 02. 2018, pp. 664–669. DOI: 10.1109/COMPSAC.2018.10315.

[22] Nighat Usman et al. "Intelligent Dynamic Malware Detection Using Machine Learning in IP Reputation for Forensics Data Analytics". In: *Future Generation Computer Systems* 118 (May 1, 2021), pp. 124–141. ISSN: 0167-739X. DOI: 10.1016/j.future.2021.01.004. URL: https://www.sciencedirect.com/science/article/pii/S0167739X21000066 (visited on 02/25/2025).

[23] Omar Yaacoubi. "The Rise of Encrypted Malware". In: *Network Security* 2019.5 (2019), pp. 6–9. DOI: 10.1016/S1353-4858(19)30059-5. URL: https://doi.org/10.1016/S1353-4858(19)30059-5.

[24] Ilsun You and Kangbin Yim. "Malware Obfuscation Techniques: A Brief Survey". In: *2010 International Conference on Broadband, Wireless Computing, Communication and Applications*. 2010, pp. 297–300. DOI: 10.1109/BWCCA.2010.85.